

Article

Performance Comparison of Native and Hybrid Android Mobile Applications Based on Sensor Data-Driven Applications Based on Bluetooth Low Energy (BLE) and Wi-Fi Communication Architecture

Aleksander Kaczmarczyk, Piotr Zając  and Wojciech Zabierowski * 

Department of Microelectronics and Computer Science, Lodz University of Technology, ul. Wólczańska 221, 93-005 Lodz, Poland; al3xkaczmarczyk@gmail.com (A.K.); piotr.zajac@p.lodz.pl (P.Z.)

* Correspondence: wojciech.zabierowski@p.lodz.pl; Tel.: +48-42-631-26-53

Abstract: In the age of smartphones and tablets, the popularity of mobile applications is rising extremely quickly. There are countless purposes for using them, such as communication, social media, news, sending emails, shopping, paying, watching videos and streams, and playing games. Google's Android is currently the most frequently used mobile system in the world. The main aim of this article is to compare native and hybrid mobile applications created for this system, taking into account the needs and challenges of designing systems using BLE and Wi-Fi. The comparison was based on mobile applications implemented in the Java 8 language and the Ionic framework. The aim was to compare the efficiency of data processing in both technologies, in order to indicate dependencies that can help in the selection of technologies in the projects under development based on BLE, Wi-Fi and sensor networks.



Citation: Kaczmarczyk, A.; Zając, P.; Zabierowski, W. Performance Comparison of Native and Hybrid Android Mobile Applications Based on Sensor Data-Driven Applications Based on Bluetooth Low Energy (BLE) and Wi-Fi Communication Architecture. *Energies* **2022**, *15*, 4574. <https://doi.org/10.3390/en15134574>

Academic Editor: Akhtar Kalam

Received: 16 May 2022

Accepted: 20 June 2022

Published: 23 June 2022

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: Android; native applications; hybrid applications; Java; Ionic

1. Introduction

We have divided the article into several parts. First of all, in the Introduction, we present the reason for our research, whilst also referring to the analyzed literature on this topic and commenting on the state of art. In the following section, we present some technological basics which are important from the point of view of knowing what we were working on. Subsequently, we present the Materials and Methods to later describe the tests carried out, and present the properly commented results.

The article ends with a discussion of the results obtained and the impact on the presented problem in conclusions.

The present world practically does not exist without smartphones, tablets and other mobile devices. In the past, producers of cell phones understood that a list of their functionalities did not have to end with calls and short messages. The term 'smartphone' gradually replaced 'telephone'. Over twenty five years ago, IBM introduced a device that could also save contacts in an address book, make notes, add events to a calendar, show emails, and handle simple games. The rapid development of cellular networks and the consequent reduction in the cost of using mobile networks, along with the increasing selection of cheap mobile devices, has resulted in many users leaving behind personal computers in favor of mobile devices. At the end of 2016, The Daily Telegraph published data which showed that mobile sites were displayed more on smartphones and tablets than notebooks and computers [1]. This was also a result of 'mobile-first' (or 'responsive web design') idea, the foundation of which was the creation of internet sites which would be available to be displayed on screens with various widths and resolutions. While the costs of the usage of

mobile phones have gone down, mobile applications have grown in popularity. Mobile-Trends.pl published, in 2017, an article with research results which showed that smartphone owners in certain countries were using even tens of mobile applications monthly [2].

Most applications can only be used on certain mobile systems, such as IOS and Android. This is because they are native applications. Therefore, a solution to this problem was sought in order to reduce application development costs. The idea of creating hybrid applications was born, in order to increase the looseness of distribution on various systems.

From the point of view of application performance, the choice of programming language is important. Therefore, such research work has already been carried out by others.

Corral, Sillitti, and Succi [3] conducted a comparative analysis of the performance of native and hybrid applications using the PhoneGap front-end for one version of the Android OS. In turn, Delía et al. [4] presented a comparative analysis of the performance achieved by mobile applications developed natively and with the use of multi-platform solutions. The authors focused on the implementation time of native applications and the different methods of creating mobile applications for many platforms at once. However, this is not the approach presented in our article.

In other publications, comparisons of performance—such as of the Android Native Development Kit (NDK) vs. Dalvik [5,6]—were made. The authors conducted various tests, such as numeric calculations (including the Ackerman recursive function), sorting (Hash, Hash2), nested loops, random number generations, and operations on strings. As a rule, it makes sense to use such a set of tests, but in our opinion, they do not fully reflect the most common and relevant operations used when working with mobile applications.

Pulasthi and Gunawardhana [7] asked which technology is better for mobile application development, but they were not backed by a real look at performance, as we do. Carvalhoi and Freire [8] referred to the subject in a similar way, in terms of the possibilities of technology in terms of functionality. Que went further in his comparison et al. [9], analyzing the paradigms of native and hybrid technologies as well as taxonomy in the work of Nunkesser and the authors [10], confirming the importance of considering these technologies.

Interesting papers in the field of the comparison of the development of native and hybrid applications in terms of the choice of tools and target systems emphasize how important is—from the point of view of the decision to choose a technology—the knowledge of the system limitations of such a choice [11,12] or framework [13] or industrial application [14] can be.

The work of Dorfer and the authors [15] on the impact of the use of hybrid technology on the CPU, memory and device batteries is worthy of note. Although this work is a very interesting study, it contains only a part of the reference to the topic we have undertaken, but it shows the need to deal with this topic in terms of hardware requirements and hardware resource consumption. Willocx and the authors [16] focused on the subject of efficiency in general in mobile applications.

Others [17–20] were also interested in applications based on hybrid solutions, based on various specific solutions and challenges they have encountered. However, this is not the broader approach we wanted in terms of our applications.

Certain aspects that can be used when choosing a technology can also be found in Phuc Huy [21] and Jobr [22].

Yet another interesting approach to the comparison of the technologies we analyzed is presented by Kho'i and Jahid, [23], and Elskowic [24], but they focus on the possibilities of these technologies, not the efficiency aspect.

During the analysis of the state of art, before making the decision to conduct our research, we also found some interesting studies showing how to migrate applications in native technology to hybrid technology [25], and the possibilities of combining these two worlds [26,27]. Although these are very interesting articles, they do not deal with the content of interest to us at all in terms of the application of these technologies to our

energy-saving projects. Even in the practical approach to the creation of mobile applications presented by Chandi et al. [28], our problem is not mentioned.

One can find many papers in which the authors analyze web-based applications (which are a subset of hybrid applications) by analyzing the performance aspects and functional limitations that also affect hybrid solutions in some way [29–34].

Those dealing with the UX aspect and the issue of application functionality are part of this type of work. We can mention El-Kaliouby et al. [35], who dealt with the conversion of code in terms of UI, and technology migration for the example of a specific portal [36] or a specific application [37], and further analysis in terms of the user interface [38].

Oliveira et al. [39] wondered whether to create applications natively or on the web in terms of energy consumption. This is another contribution to the discussion of how important this choice is. However, this approach does not include our case, i.e., the use of hybrid technology. The strategies of developing mobile applications are analyzed by Wilcox et al. [40] in terms of performance.

Zhou even shows the use of native solutions in terms of online experiments, only with the use of mobile applications, in this case with the use of React [41].

Other authors considered the improvement of the capabilities of cross-platform applications in mobile applications [42], mentioned the alternative of progressive applications to native solutions [43], and focused in greater detail on the practical possibilities of using web-based applications in the hybrid version [44].

All of these described solutions emphasize and show the importance of the analysis of the technological differences of the mentioned technologies. However, regardless of all of the cited publications, it must be clearly said that none of the teams focused on the problem formulated by us for our needs. Although they came out of our needs, the results will surely perfectly complement the knowledge base in this field.

The tests in this publication generally refer to the qualities of the chosen application type.

Our team also conducted comparative research for various technologies in order to be able to choose the best solution when planning an application that analyzes and collects data from a network of sensors [45].

In our case, this is very important, because the smartphone is the basis for supporting the sensor network in some of our medical projects, where a large number of sensors are installed on the patient's clothes or body. In addition, the application should communicate with external sensor networks (Figure 1) which are also based on BLE beacons.

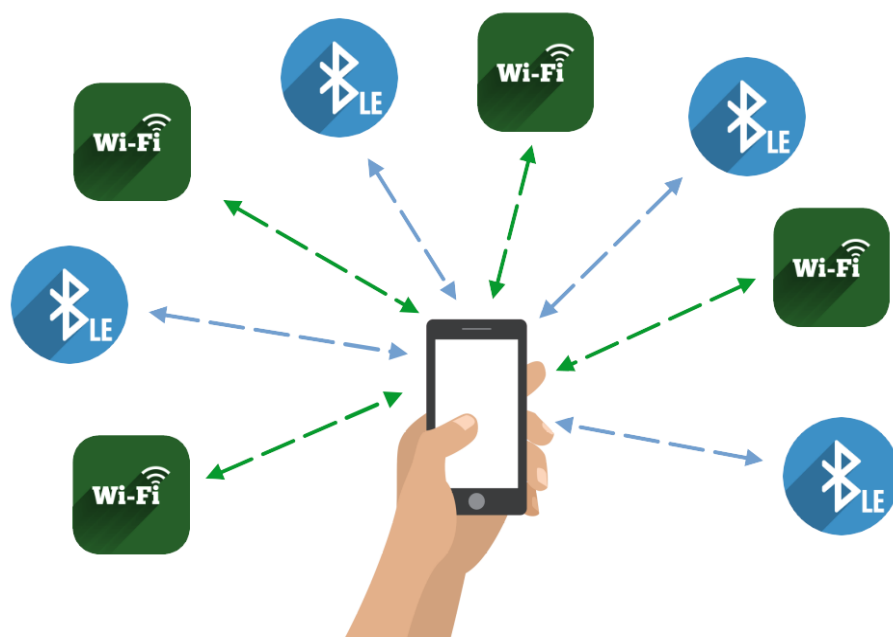


Figure 1. Project data transmission layout.

This is a very schematic and universal approach to the architecture used in our projects, with particular emphasis on devices such as BLE and Wi-Fi, which in our solutions have power consumption problems, and the optimization of their energy expenditure had to be considered.

Communication takes place both via Wi-Fi, providing data for analysis, and via BLE beacons in order to position the patient. Despite the use of low-energy BLE and the Wi-Fi modules themselves, the processing time—and therefore the consumption of energy resources of communication sensors—is important. It is known that if the processing takes a longer time, it can lead to overflows and higher energy consumption, e.g., beacons. Likewise, a greater consumption of the resources of the central unit itself can lead to delays, and thus also to additional energy losses. This is why it is so crucial to choose the right programming technology due to the specifics of the project.

Therefore, in a situation where the reaction to the received data from sensors must be rapid, or even immediate, not only are sensors with appropriate accuracy and speed important but also the application cannot introduce an unexpected delay. According to our experience from previous projects and discussions with other teams, the choice of programming technology was usually based on subjective feelings and habits, or even came down to implementing a language because someone knew it beforehand. Our research, which arose during the academic research of the previously mentioned medical projects, gives the opportunity to make an appropriate choice of mobile application technology based on the very specific parameters described in this article, according to requirements.

The undoubted contribution of the article, and in fact the research that was the basis for its writing, is to conduct a comparison that gave us an answer regarding which of the selected technologies is better suited for creating applications in the situation of working on the edge of resources, in particular in terms of energy use. Because in our solutions we often use sensor networks that must communicate via either Wi-Fi or BLE modules with an application that collects and processes data, we wanted to check how to optimize our projects in terms of energy. As shown in Figure 1, in a situation in which there are many such sources, often with a limited power supply, the differences in energy efficiency of each of these technologies can be and are crucial for the appropriate lifetime of the system and its reliability.

It is in our article that we showed the basis of our research. The results themselves, together with the discussed results, are also useful for anyone who may not have thought about this yet when looking for energy savings elsewhere. Our experience on the borderline of designing sensor networks and creating applications indicates this area, i.e., the use of an appropriate technological model of the application to obtain the energy yield here. Such optimization, in some cases, may even determine the effectiveness of the entire project, or may simply reduce costs when choosing cheaper system components, due to the savings obtained.

In the description of ready-made solutions in this field, we showed that, first of all, the topic is important because comparisons of this type of technology have already been carried out; at the same time, we have shown that—so far—no one has approached the comparison of these technologies in terms of reducing the burden on resources, and thus in terms of the reduction of energy consumption. Our programming and design experience has resulted in this idea, the results of which have been successfully implemented in our various projects. We have made a thesis that the choice of technology to write an application may reduce the load on the device resources on which the application runs, while at the same time having an acceptable level of data processing. This, in turn, without additional evidence, will reduce the energy demand of the system components. For example, shorter processing times mean shorter communication times with system components, and therefore shorter times when they must operate at an increased or the maximum level of energy expenditure. Therefore, contrary to appearances, the smaller number of references in the literature is not a disadvantage; it proves the innovativeness of our approach. Our comparison is not a comparison in itself, as can be found in other articles. It was made for a specific purpose:

to reduce energy expenditure by selecting the appropriate technology that will reduce the energy consumption of the system.

2. Background and Technology Description

2.1. Android System

Although it would seem that everyone knows what Android is because they have it in their phones, it is necessary to say a few words about its popularity. Due to the long history of its development, which resulted in a lot of flexibility and more and more new versions, it is also a great opportunity to optimize the system itself.

What is more, this popularity means that the system can also be found on many devices, not only telephones but also everyday devices such as TV sets or refrigerators.

All of this prompted us to choose this system to analyze the problem posed in accordance with the thesis.

Android was created in 2003. Android Incorporation was founded in the United States of America. The main motto of its founders was to develop smart mobile devices “that are more aware of its owner’s location and preferences” [46]. The company was bought two years later by Google, which started to work on its own mobile devices and a mobile system. They introduced their first smartphone with Android in 2008, but it did not revolutionize the market. The first version of Android had some default applications installed. Many improvements have been published since the beginning of the system. Each version offered new possibilities and the optimization of hardware operations, user interfaces, and above all, security. Due to enormous support of Google, Android has become the most important and popular system for mobile devices and more.

2.2. Native vs. Hybrid Applications

Native applications are created for a specific operating system of a mobile device. In the case of Android, this is Java or Kotlin [47]. For iOS, this is Swift or Objective-C. The authors prepared tools allowing them to build applications for those systems. Due to differences concerning the mentioned languages and user interface elements, mobile applications are often implemented in two versions.

Native applications have their own advantages, primarily including fast running, high efficiency, and easier access to sensors and modules of mobile devices (e.g., accelerometer and GPS). The undoubted advantage is also their easier distribution in the app store. These factors especially encourage global publishers to choose native solutions in the case of more advanced applications.

Hybrid applications use the WebView component. Thanks to this, the application combines the advantages of a web application and a native application. The structure of such applications is based on HTML (HyperText Markup Language) with interface elements, CSS (Cascading Style Sheets) with definitions of elements’ styles, and JavaScript files with the code for the handling of specific events and user actions. Hybrid applications can be run on various operating systems of mobile devices. This saves one from having to implement the application on each system separately.

The mentioned option of building one project for many platforms has a lot of advantages, especially the cutting of financial costs (a lower number of programmers, and lower license and test device costs) and time (which causes the much faster publishing the application). Hybrid applications can also access the device modules, but they need external plugins in order to accomplish this. The user interface is built in a simpler way; however, using proper style definitions can help to achieve a very similar effect to native applications. Hybrid applications might be a better solution for the creators of simple and easy-to-use mobile applications of which the features are not based much on device modules.

A short summary of these differences is given in Table 1.

Table 1. Summary of the differences between hybrid and native technology.

	Hybrid Applications	Native Applications
Advantages	• Less expensive development process—saving resources (one code for both platforms means employing one development team) • Easy application distribution • Built-in web browser • Support for built-in device functionality	• Ability to operate offline • Possibility of better use of all device functions (such as push notifications, sensors, camera) • Greater possibilities and sophistication of the created functionalities • Clear code architecture • The use of modern languages (e.g., Swift, Kotlin) that are easy to develop • Better testing tools (easier error detection)
Disadvantages	• Usually slower performance than the native application, the code is not always compiled to the native code of the application • Frequent changes in the JavaScript ecosystem • Less tailored User Experience	• Costly development process due to hardware differences • High maintenance and application maintenance costs

2.3. Java 8 vs. Kotlin

The position of Java in the rankings of the most popular programming languages has consistently been very high in recent years. The history of Java started in 1995 with James Gosling from Sun Microsystems. One of the most important current uses of Java is the development of mobile applications for Android [48] systems.

Java 8 can be used to build mobile applications which can be run on Android 7.0 or newer versions. In this version of Java, the authors introduced functional interfaces which contain only one abstract method. They usually have specific purpose, e.g., to predicate with the `test()` method to check the logical value of expression based on the given arguments, or `Comparator` with the `compare()` method for returning the result of the comparison of two variables. Functional interfaces are often used in lambda expressions, which shorten the code of anonymous functions and simplify the iteration and filtering elements of a collection. Method references are also a new feature in Java 8, and are often used in lambda expressions. There is also streams API—streams allow one to execute operations on collections more efficiently. Interfaces may now contain static and default methods. The authors added new date and time APIs as well. The new `Optional` class should be mentioned too—its purpose is to prevent the occurrence of `NullPointerException`.

New features in Java 8 are generally appreciated by programmers. Due to those changes, building applications in this language seems to be much easier. This update is considered to be the most important in the history of Java.

Developers appreciate the new improvements in Java 8. These changes facilitate the development process, which in the opinion of the development environment makes Java 8 the most important update ever.

Kotlin is another programming language which might be applied in native mobile application projects. It was developed by JetBrains, and is based on the Scala language. In 2017, Google announced Kotlin as an official programming language for the Android system, and two years later it was presented as a preferred language. Due to the fact that Kotlin can run code written in Java, many Java programmers have decided to migrate their projects to Kotlin.

Kotlin has features of both object-oriented and functional programming languages. It is a statically typed language; therefore, many errors can be found during the compilation process. Its authors claim that code written in Kotlin should be 40% shorter than code written in Java [47], which can be noticed even in the simplest projects. This also means that code written in Kotlin should have higher quality. The code does not require semicolons, contrary to the Java language. Significant code reduction is an effect of classes with data attributes (with fields and their values defined in a single line). One of the most important

advantages is a much easier way of avoiding `NullPointerException`, especially due to nullable and non-nullable types, and also due to an operator called the Elvis operator checking the null safety of the values. Due to the lazy-loading feature, the startup time of Kotlin applications is short. Many programmers appreciate extension functions which can bring new functionality to already existing classes and libraries. Similarly to Java 8, Kotlin allows the use of collections and streams API, even in a more flexible way.

Kotlin is a cross-platform language; therefore, it can be applied in projects with different purposes. Mobile applications built in Kotlin may also run on iOS. Moreover, it can be used in both back-end and front-end applications. Kotlin's developers are also the creators of extremely popular and advanced IDEs (Integrated Development Environments) such as IntelliJ IDEA and Android Studio, which provide a lot of useful tools and features for programmers. This programming language is still trending, and might soon overcome Java in terms of popularity.

2.4. The Ionic Framework

Ionic was published at the end of 2013 as a web user interface framework [49–51]. The term 'framework' defines a reusable set of connected elements which is used to cut the costs of building software and improve its quality.

The first version of Ionic was based on the AngularJS platform for the creation of web applications. The main motto of Ionic authors was to deliver free programming tools in the rapidly growing field of mobile applications.

The revolutionary second version of Android had a big impact on the next version of Ionic. Ionic 2, as well as Angular 2, are based on the TypeScript programming language designed by Microsoft. TypeScript is a superset of JavaScript, and the compilation of this language consists in the simple translation of the code to JavaScript.

Variables in TypeScript need to be typed, which is one of the most important differences between this language and JavaScript. Programmers may also create classes (also containing optional fields). An interesting option is the possibility to make combinations, unions and aliases of types. Code written in TypeScript should be easier to read and less vulnerable to errors than JavaScript.

Ionic applications are based on a modular project structure. A module is a set of related components which are responsible for a certain functionality. A component contains a TypeScript file with code for handling events and user actions, a HTML file with a defined layout with user interface elements, and a CSS file with style definitions. A component may be built of other components. Services have a crucial role in delivering some important functionalities to components, especially in communication with a server through the `HttpClient` module.

Ionic uses Apache Cordova plugins. Thanks to this, it is possible to access devices and sensors, e.g., cameras and acceleration sensors. However, this does not limit the use of other third-party plug-ins.

3. Materials and Methods

For the purpose of comparing hybrid and native Android apps, a mobile application was developed in two versions. The native version was created in Java 8, and the hybrid version was made using the Ionic framework. Several tests were conducted concerning both the efficiency and implementation of those applications.

3.1. Tested Application

The developed application made it possible to publish and review promotions in the observed stationary and online stores. The application needed to meet requirements such as:

- an overview of all of the deals and a single deal with a title, the nickname of the author, the price before promotion, the price after promotion, the score (the difference of users' votes), and a description and optional photo;

- choosing the top or newest deals;
- filtering promotions by the user localization or searching phrase;
- checking exchange rates;
- registration;
- signing in by an user;
- adding and modifying a deal;
- commenting on a deal;
- voting for a deal;
- checking a user profile;
- modifying user data.

The application should also have a user-friendly interface, be easy to use, and work correctly on devices with various sizes. It ought to communicate with API rapidly, and should process data without any interruptions. The application needs to have fluent transitions between its sections. Moreover, its project should be scalable and easily managed by the programmers.

3.2. Test Rig

Both the mobile applications and server application were designed, implemented and built on a PC with the following specifications:

- CPU: Intel i5-2520M 2.5 GHz;
- RAM: 8 GB (dual-channel, 1666 MHz);
- Hard drive: 120 GB SSD;
- Operating system: Windows 7 64-bit.

The tested applications ran on an emulated Android Virtual Machine with following specification:

- Model: Google Pixel 2XL;
- CPU cores: 2;
- RAM: 1GB;
- Operating system: Android 8.0.

3.3. Comparison Assumptions: Project Structure

Firstly, the structure of both projects was compared. This comparison was based on the number of directories and files containing source code. The data was obtained after the analysis of the properties of both projects in the operating system.

3.4. Comparison Assumptions: Build Time

Building a mobile application in Android systems consists in compiling the source code and creating an APK (Android Package Kit) file. It is this file that allows installation on a given mobile device. The time was checked in a terminal after the building processes of both applications.

3.5. Comparison Assumptions: Speed of Running

The speed of running is crucial in the case of mobile applications. It was tested on the emulated device. First of all, the time of starting the application was checked based on the logs. The overall time included the startup of the process, the initialization of the objects, the creation and initialization of the activity, the initialization of the layout, and the displaying of the application.

Additionally, tests of transitions between application elements were also performed. The speed in this element of operation improves the use of the application interface. The times of transition from the user profile view to the login screen were checked in a console.

Moreover, this article presents the test results for data processing times that were downloaded from the REST API. This is crucial for the proper construction of a responsive mobile application. In the case of the native application, the authors of this article used the

Retrofit library, which significantly simplifies this process due to using proper annotations in code and handling errors. The application built in Ionic has the HttpClientModule, which is responsible for this communication. It also provides error handling and casting data to given objects. Moreover, it allows operations on data streams and observables. The times were counted between the two flags. These were the moment of sending the request and the end of loading the full data in the emulator mentioned before. The server with REST API was run locally.

3.6. Comparison Assumptions: Usage of Hardware Resources

The usage of hardware resources was tested as well. Nowadays, mobile devices have gigabytes of operating memory and storage; therefore, this is not the most important factor. Nevertheless, programmers need to avoid memory leaks and application hangouts. The statistics were checked in the Developer Options in the emulated smartphone.

3.7. Comparison Assumptions: Amount of Code

The amount of source code in both projects was also compared. Generally, having less code positively affects the readability and maintenance; also, it often allows programmers to save more time. Lines of code in given files were analyzed.

3.8. Comparison Assumptions: External Libraries

Multiple mobile application projects use dependencies from external libraries. They are generally free to use and have open sources, helping programmers to build their applications and apply 'clean code' rules. In the case of the tested applications, there were several usages of third-party libraries. The method of their installation was compared.

3.9. Comparison Assumptions: User Interface

The author of this article compared the implementation of the user interface as well. Elements such as buttons, labels, and inputs are commonly used in mobile applications, and their availability was compared. Furthermore, the final design was analyzed.

3.10. Comparison Assumptions: Storing Data Locally

Sometimes, mobile applications need to store their data in the local memory. For example, this could be information about currently logged users. The tested application stores the JSON Web Token, which is responsible for user authorization. The method of storing such data was compared.

3.11. Comparison Assumptions: The Usage of Sensors

Mobile devices are built of many important modules, such as cameras and GPS. As mentioned before, native applications have direct access to them, whereas hybrid applications rely on additional plugins to process the data from those modules. The tested application needed to find the user's localization to show offers from local stores. The method of obtaining this data was analyzed.

3.12. Comparison Assumptions: Documentation and Support

From a programmer's point of view, it is meaningful to have support from the programming language community and developers. There is no denying that some problems might occur during the process of mobile application development, and programmers may not be able to solve them by themselves. The availability of documentation and support was compared.

4. Results

4.1. Project Structure

Native application projects have a standard Java project structure. Using the project wizard in the Android Studio environment, it is possible to create a simple application very

easily. The default structure is based mainly on directories such as `src/main/java` (where Java code is stored) and `src/main/res` (which contains files with defined layouts, images, icons, menus, styles, colors or labels).

Hybrid applications built in the Ionic framework have a structure which is very similar to projects created in Angular, one of the most popular web application frameworks. Crucial files are stored in the `src` directory, including `index.html` as a start page, `global.scss` with global style definitions, and `main.ts` with the code of the application's initial loading. All of the static resources are stored in `src/assets`. Modules with their components and services are available in `src/app`. Components are grouped into separate directories.

Figure 2 presents data concerning the structure of both projects, which did not contain files generated in the building process or files related to specific development environments. Both projects used a very low amount of disk space. Hybrid application projects have a higher number of files and a lower number of directories. This may be related to the mentioned grouping of the components into separate directories. The structures of these projects were very light, and could be easily stored in repositories.

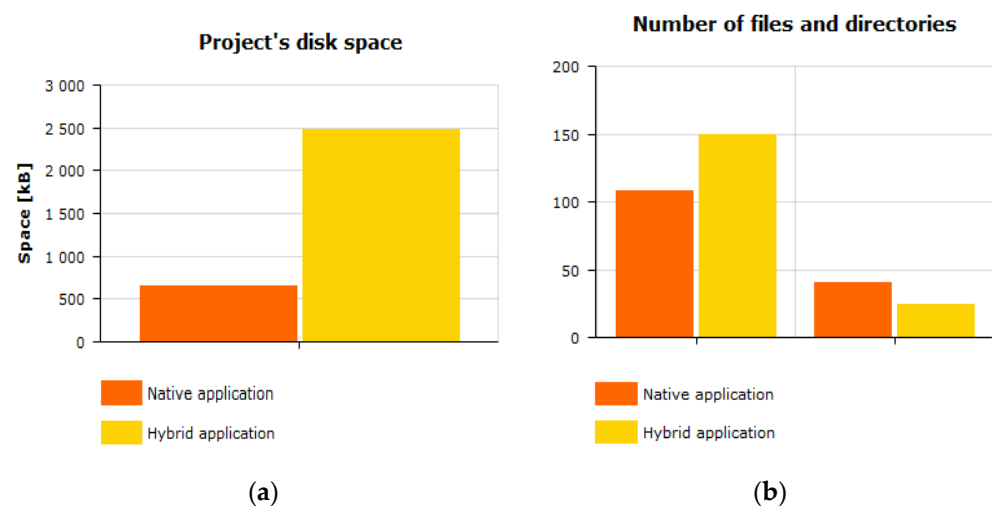


Figure 2. Project structure: (a) project's disk space; (b) number of files and directories.

It is obvious that if you can limit the number of files and the disk space usage, this allows you to reduce the memory resources needed by the device for the application itself. This means that, without increasing the cost of the device, more disk space or memory remains for the buffering of the received data from the sensors.

4.2. Build Time

In the case of the native application, this process is performed by Gradle. It may be started in Android Studio IDE after choosing the Make Project or Build APK(s) options in the Build section in the top menu. Alternatively, a programmer can execute the command `gradlew assembleRelease`. The building of hybrid applications in Ionic is managed by the Node Package Manager (NPM) delivered with Node.js platform. In order to prepare an APK file, two commands need to be executed: `ionic cordova build android` and `cordova build android`. As shown in Figure 2, building a hybrid application was on average twice as slow as building a native application (Figure 3). The difference would probably become even larger in the case of more advanced projects.

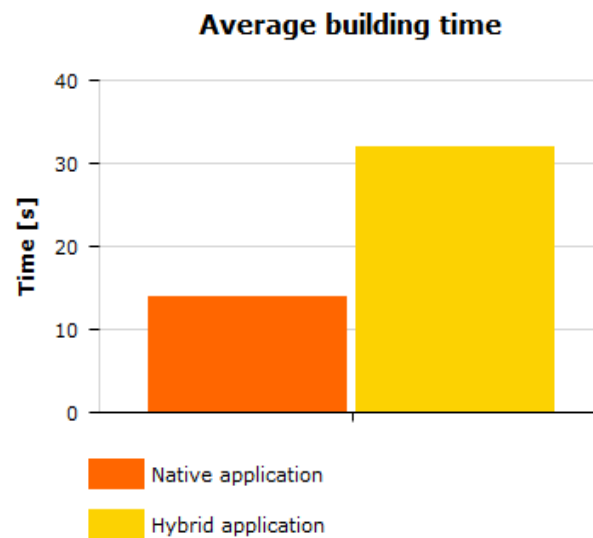


Figure 3. Average building time.

The obtained results are in line with the assumptions resulting from the specificity of the technology. From the point of view of the application itself, they do not matter, but they are crucial when creating the project, and possibly when implementing the subsequent corrections.

4.3. Speed of Running

The native application startup time was faster (Table 2). This may be due to the fact that the hybrid application had to load a large number of scripts. The times differed by about a second, which might be noticeable when the user starts the application.

Table 2. Time taken to start the applications.

Number of Test	Time of Starting Native Application [s]	Time of Starting Hybrid Application [s]
Test 1	1.359	2.682
Test 2	1.276	2.357
Test 3	1.237	2.366
Test 4	1.224	2.247
Average	1.274	2.413

The times of the transitions from the user profile view to the login screen are presented in Table 3. In the native and hybrid applications, the transitions were rapid. However, after loading the scripts at the beginning, the hybrid application was slightly better in transitions.

Table 3. Time taken to transition between the user profile and login screens.

Number of Test	Time of Transition in Native Application [ms]	Time of Transition in Hybrid Application [ms]
Test 1	174.0	50.0
Test 2	176.0	43.0
Test 3	156.0	33.0
Test 4	206.0	36.0
Average	178.0	40.5

Processing data from REST API are shown in Tables 4–6. Regarding the hybrid application, the longer it processed data, the more there were. The native application clearly won here.

Table 4. Time taken to process the data from REST API (response with 10 objects).

Number of Test	Time of Processing Data in Native Application [ms]	Time of Processing Data in Hybrid Application [ms]
Test 1	39.0	75.0
Test 2	35.0	47.0
Test 3	29.0	77.0
Test 4	39.0	69.0
Average	35.5	67.0

Table 5. Time taken to process the data from REST API (response with 50 objects).

Number of Test	Time of Processing Data in Native Application [ms]	Time of Processing Data in Hybrid Application [ms]
Test 1	50.0	158.0
Test 2	42.0	173.0
Test 3	51.0	138.0
Test 4	57.0	173.0
Average	50.0	160.5

Table 6. Time taken to process the data from REST API (response with 100 objects).

Number of Test	Time of Processing Data in Native Application [ms]	Time of Processing Data in Hybrid Application [ms]
Test 1	78.00	277.00
Test 2	58.00	238.00
Test 3	72.00	245.00
Test 4	49.00	222.00
Average	64.25	246.25

The hybrid application loses to the native application when it comes to launching the native application. This may affect the operation of the system in a remote restart situation. However, already at the time of loading, the cross-plant trafficking in the case of the hybrid application gives it an advantage, which is more significant from the point of view of the system operation. Application restart happens rarely, and the natural movement between tabs in the user process happens often. This, of course, affects the access times to the application and thus the energy expenditure of the communicating system components.

Tables 4–6 show that with the increase in the number of objects, i.e., the data load, the processing time is in favor of the native application, which was to be expected. However, another important observation follows from this: with a small amount of data, the processing times are almost identical. This means that when we are dealing with systems where data is received at larger time intervals and in a small volume, there is practically no difference between these technologies, which in turn—in relation to other advantages of the hybrid application—leads to the conclusion that it is better to use this technology then.

4.4. Usage of Hardware Resources

In both cases, one can observe that the applications do not take up a lot of disk space or RAM, as seen in Figure 4. This can certainly differ depending on the specific application. It is important, however, that it was the hybrid application that needed more of these resources. This means that native applications have better optimization in the Android system.

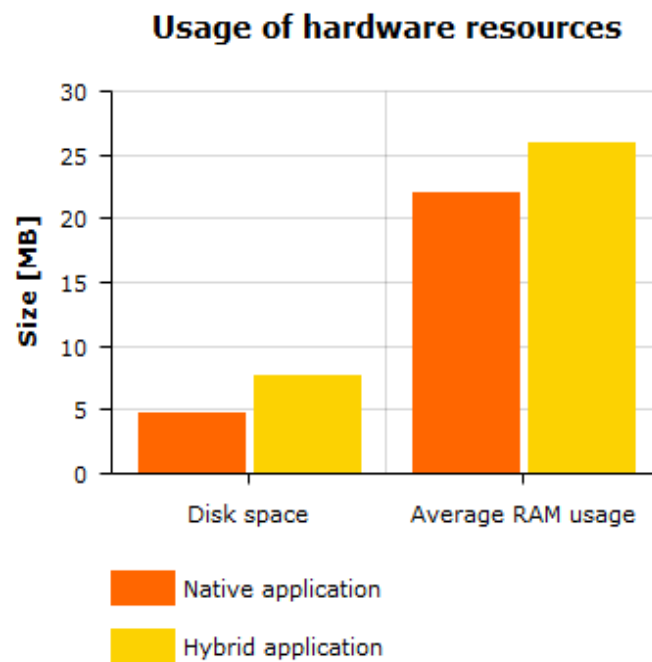


Figure 4. Usage of hardware resources.

Less use of hardware resources should be understood as a potential reduction of system development costs through the use of cheaper components, but is also crucial from the point of view of energy efficiency. The fewer resources are occupied, the less energy is needed to power the system, such that working on the edges of energy resources creates space for optimization.

4.5. Amount of Code

The native application project contains XML (Extensible Markup Language) files with layout definitions and applied styles (sizes, colors, margins, paddings etc.). The hybrid application project contains HTML files with user interface components and SASS files with their styles. HTML files might include variables, loops, and conditional blocks, which help to avoid irrelevant duplicates and reduce the amount of code (see the comparison in Figure 5a). Furthermore, the project built in Ionic had much less code in files with controllers (event handling and business logic) and data models (as seen in Figure 5b). Contrary to the native application, there is no need to define objects representing user interface elements and getters/setters for data models. Data binding provides flexibility and readability in the case of the hybrid application projects. In conclusion, it should be easier to develop and maintain hybrid applications than native ones.

The amount of code may seem insignificant from the point of view of the application's operation. On the one hand, the amount of code needed to obtain functionality has an impact on the speed and the costs of system development, but it can also affect the consumption of resources and thus the issues related to energy optimization. In this case, the hybrid approach clearly wins.

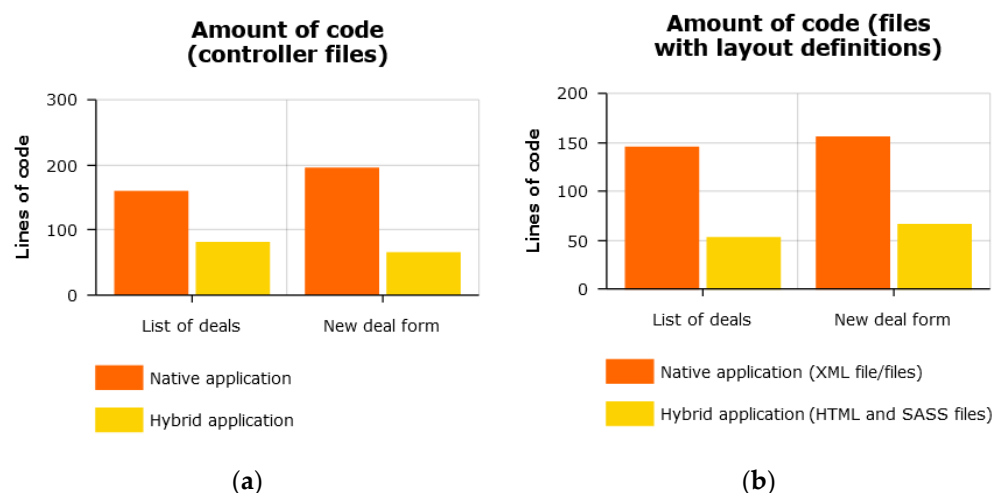


Figure 5. Amount of source code in: (a) controller files; (b) files with layout definitions.

4.6. External Libraries

The native application was dependent on libraries such as JWT (user authorization), ButterKnife (simpler managing user interface elements), Retrofit (communication with REST API), and Glide (operations on icons and images). The hybrid application had many more default required libraries (generally connected to Angular, Ionic and Apache Cordova); however, they were sufficient to fulfill the tested application requirements. In both cases, the use of third-party libraries was extremely easy due to only adding the name and version number of a given library to a proper file (build.gradle or package.json) and executing specific commands (gradlew build or npm install).

In this case, it should simply be stated that this mainly affects software engineering and related expenditure.

4.7. User Interface

The native application has direct access to all of the user interface components available in the Android system. The hybrid application built in Ionic could use multiple elements which had almost the same use as those available in the native one (see the comparison presented in Table 7). This means that the final design of a mobile application can be very similar in both cases (presented in Figure 6). Therefore, the user interface should not be a determinant of the choice between native and hybrid applications.

It can be seen that regardless of the choice of the discussed technologies in this study, the visual effect is similar. This is a very important premise, given that in other aspects you should choose a hybrid version, you don't have to wonder if the user interface functionality will be sufficient. This is very important.

Table 7. User interface components.

Purpose	Native Component	Component Available in Ionic
Label	TextView	ion-label
Input	EditText	ion-input
Button	Button	ion-button
Floating button	FloatingActionButton	ion-fab-button
Select from list	Spinner	ion-select

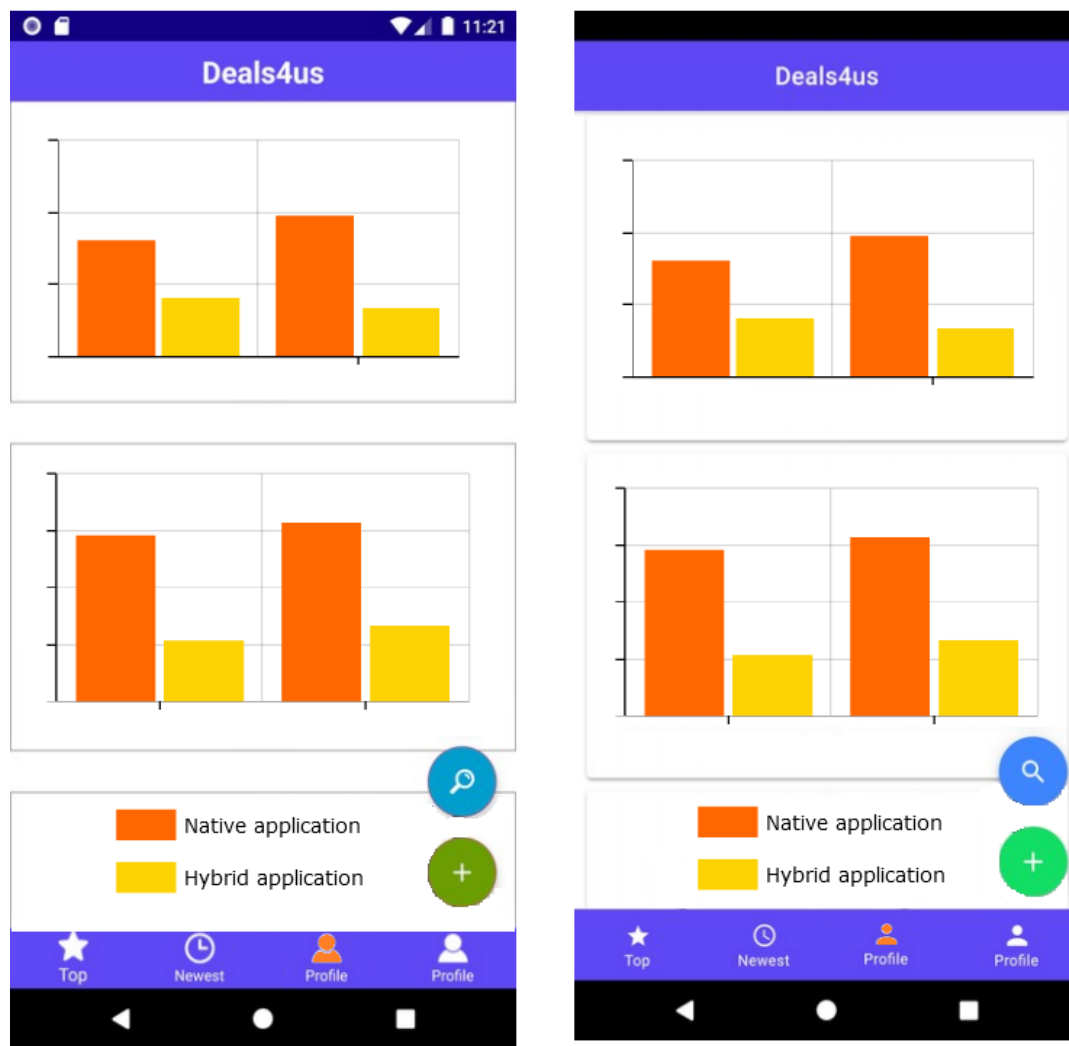


Figure 6. User interface (list with deals) in (left side) the native application and (right side) the hybrid application.

4.8. Storing Data Locally

The native application uses a `SharedPreferences` object to contain key-value pairs, and uses an `Editor` object to modify them. The hybrid application may read and update `LocalStorage`, which is commonly used in web browsers. Storing local data is simple and similar in both scenarios.

4.9. Usage of Sensors

In the case of the hybrid application, two lines of source code were enough to estimate the geographic coordinates. What may seem surprising is that it was more difficult to find the user localization in the case of the native application. It was necessary to create a class implementing the `LocationListener` interface and methods which checked privileges and the possibility to process this data. Moreover, the geolocalization was less accurate than in the hybrid application. Therefore, using data from device modules is not a strong argument in the discussion of native and hybrid mobile applications.

As you can see, the smaller the amount of code needed to obtain information from the sensor in the hybrid solution, the smaller the amount of resources needed for the processing and storage of the code. However, this was commented on in Section 4.5.

4.10. Documentation and Support

The Android system and Ionic framework have detailed documentations published online. Due to the existence of multiple tutorials, it is easy to start working on mobile application projects. What is more, many IDEs provide helpful tools for programmers. Both native and hybrid application programmers are active on forums such as StackOverflow, one of the most popular platforms where programmers help each other to solve shared issues. The huge number of questions concerning the building of applications for Android systems shows how popular native applications are. Over one million questions were connected to Android [48], and tens of thousands concerned the Ionic framework [51]. This is a proof that hybrid applications are still much less popular than native ones; however, their popularity should be still growing.

5. Discussion

The conducted research confirmed some of our assumptions, but also gave an answer to the question of how, regarding the choice of technology, certain compromises resulting from the choice affect the work of the entire system. The collected information from the tests is summarized in Table 8.

Table 8. Advantages and disadvantages of the tested technologies based on the conducted research.

Comparison	Hybrid Application	Native Applications
Project structure	More files and disk occupation	Less files and disk occupation
Build time	Slower	Two times faster
Speed of running—starting app	Slower	Two times faster
Speed of running—transition	Even three time faster	Slower
Speed of running—from Rest API	Slower	Two times faster, and grows due to number of response objects
Hardware resource usage	More	about 20% less
Amount of code	Two times less	More
Amount of code files	Even three times less	More
External libraries	Similarly, it depends on the design	similarly, it depends on the design
User interface	Similar, the differences are generally insignificant.	Similar, the differences are generally insignificant.
Storing data—locally	Simple and similar in both scenarios	Simple and similar in both scenarios
Usage of internal sensors	Easier—less amount of code needed	More complicated
Docs and support	Less popular but growing	More popular

The project structure was compared at the start—the differences between the two are rather irrelevant. The number of files and directories and the disk space taken up were very similar; therefore, these cannot be important criteria of this comparison.

The build times should be taken into consideration during the development process. The conducted tests showed that the building of native applications was twice as fast. However, due to live-rebuilding processes, the first build time is not very crucial.

Generally, the most important factor is the speed of running. Users often appreciate responsive, rapidly working applications. The advantage of native applications seems to be significantly noticeable. The faster starting time by one second can be crucial for users who want to use an application for a few seconds only. Furthermore, the processing of data from external sources occurs in most applications. Tests were conducted for simple objects, and their number did not exceed one hundred. Native applications as much as four times faster than hybrid applications. For more advanced objects and a higher number of them, the difference might be even bigger.

The efficiency of mobile devices is increasing very quickly nowadays. This does not mean that the usage of resources is not important for developers. The tests indicated that hybrid applications use slightly more RAM and disk space. The differences could be more noticeable in the case of more advanced projects. In the case of simpler applications, this factor is rather irrelevant (assuming that the applications do not cause memory leaks).

User interfaces can be implemented in very similar ways. The application built in Ionic has components with the same purposes as the native ones. The screenshots from both applications show their almost identical layouts. This means that the user interface implementation is not a strong factor when choosing between native and hybrid applications.

From a programmer's point of view, there are many other important criteria which lead to making a decision about whether a project should be built using a native or hybrid solution. Our research shows that the usage external libraries, the storing of local data, and the usage of sensors did not differ significantly. Native application projects could be more spacious based on the analyzed amount of code in the case of both projects. However, this should not be a determinant of fulfilling clean code rules.

Finally, hybrid applications are still relatively new solutions, and the process of their development will probably continue over the next few years. Many problems could be solved, and performance might be optimized. Taking this into consideration, it might be said that hybrid applications can turn out to be a better choice from both the user's and programmer's point of view.

Among the readers, a question or doubt may arise as to whether the obtained results are susceptible to falsification. Of course, it must be remembered that in the case of this type of research, the specific numbers are less important than the mutual relations between the obtained values. This is due to the fact that each system is designed slightly differently, on slightly different components; therefore, the obtained numerical results of the times in each test, for each application will certainly be different. The obtained results and the conclusions resulting from them should be treated as general dependencies in order to help in the decision to choose the technology, and were treated as such from the very beginning. Therefore, the thesis was confirmed, and it was shown how the examined aspects of both technologies can have an impact on the energy and cost optimization of the project.

6. Conclusions

By comparing native and hybrid mobile applications for Android systems, thanks to the tests carried out, we proved that native applications have higher performance. The comparison also showed that hybrid applications are easier to build and maintain. The speed of operation in both cases was similar, but the processing times of the data from the REST API responses show the advantage of the native application. However, it is important to note that—in many cases—the hybrid application was not inferior to the native application. The possibility of designing mobile applications with Ionic for many systems should be considered by publishers despite its slightly worse statistics. If performance is the key, one should usually use an application dedicated to a given system, i.e., native to Android or iOS.

The results of our research are universal, and so will be helpful for anyone who wants to develop an application that collects and processes data from any sensor network. In order to make a final decision, apart from the obtained results, only the information about the operations most frequently performed by the designed mobile application is needed.

This article provides answers to the questions not yet supported by the literature, and makes a significant contribution to the development of mobile applications.

The approach to the examination of some of the implementation differences presented in the article was based on specific problems encountered by the authors in the implementation of the projects discussed in the introduction. A similar approach to the comparison of the discussed technologies has not been found. Thanks to the research work, it was possible to obtain material supporting decisions when choosing a programming technology for the implementation of distributed systems based on sensors, BLE beacons and Wi-Fi trans-

mitters operating in difficult power conditions. The obtained and discussed results will certainly help anyone who is faced with a similar choice related to the choice of technology.

Author Contributions: A.K.: data curation, investigation, writing (original draft); P.Z.: supervision, writing (review and editing); W.Z.: conceptualization, supervision, writing (review and editing). All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Not applicable.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Mobile Web Usage Overtakes Desktop for First Time. Available online: <https://www.telegraph.co.uk/technology/2016/11/01/mobile-web-usage-overtakes-desktop-for-first-time/> (accessed on 10 October 2021).
2. W 2017 Roku Rynek Aplikacji Mobilnych Miał Się Świetnie. Available online: <https://mobiletrends.pl/raport-2017-roku-rynek-aplikacji-mobilnych-mial-sie-swietnie/> (accessed on 10 October 2021).
3. Corral, L.; Sillitti, A.; Succi, G. Mobile Multiplatform Development: An Experiment for Performance Analysis. *Procedia Comput. Sci.* **2012**, *10*, 736–743. [CrossRef]
4. Delía, L.; Galdamez, N.; Corbalan, L.; Pesado, P.; Thomas, P. Approaches to mobile application development: Comparative performance analysis. In Proceedings of the Computing Conference, London, UK, 18–20 July 2017; pp. 652–659.
5. Lin, C.; Lin, J.; Dow, C.; Wen, C. Benchmark Dalvik and Native Code for Android System. In Proceedings of the 2011 Second International Conference on Innovations in Bio-inspired Computing and Applications (IBICA), Shenzhen, China, 16–18 December 2011; pp. 320–323.
6. Fikri, A.; Presekal, A.; Harwahu, R.; Sari, R.F. Performance Comparison of Dalvik and ART on Different Android-Based Mobile Devices. In Proceedings of the 2018 International Seminar on Research of Information Technology and Intelligent Systems (ISRITI), Yogyakarta, Indonesia, 21–22 November 2018; pp. 439–442.
7. Pulasthi Dhananjaya Gunawardhana, L.K. Native or Web or Hybrid which is better for Mobile Application. *Turk. J. Comput. Math. Educ.* **2021**, *12*, 4643–4649.
8. Carvalho, L.P.; Freire, A.P. Native or Web-Hybrid Apps? An Analysis of the Adequacy for Accessibility of Android Interface Components Used with Screen Readers. In Proceedings of the IHC 2017, Joinville, Brazil, 23–27 October 2017.
9. Que, P.; Guo, X.; Zhu, M. A Comprehensive Comparison Between Hybrid and Native App Paradigms. In Proceedings of the 2016 8th International Conference on Computational Intelligence and Communication Networks, Tehri, India, 23–25 December 2016.
10. Nunkesser, R. Beyond Web/Native/Hybrid: A New Taxonomy for Mobile App Development. In Proceedings of the 2018 ACM/IEEE 5th International Conference on Mobile Software Engineering and Systems, Gothenburg, Sweden, 27 May–3 June 2018.
11. Jiang, S. *Comparison of Native, Cross-Platform and Hyper Mobile Development Tools Approaches for iOS and Android Mobile Applications*; University of Gothenburg Chalmers University of Technology Department of Computer Science and Engineering Göteborg: Gothenburg, Sweden, 2016.
12. Vilček, T.; Jakopc, T. Comparative analysis of tools for development of native and hybrid mobile applications. In Proceedings of the MIPRO 2017, Opatija, Croatia, 22–26 May 2017.
13. Nawrocki, P.; Wrona, K.; Marczak, M.; Sniezynski, B. A Comparison of Native and Cross-Platform Frameworks for Mobile Applications. *Computer* **2021**, *54*, 18–27. [CrossRef]
14. Biørn-Hansen, A.; Grønli, T.-M.; Ghinea, G.; Alouneh, S. An Empirical Study of Cross-Platform Mobile Development in Industry. *Hindawi Wirel. Commun. Mob. Comput.* **2019**, *2019*, 5743892. [CrossRef]
15. Dorfera, T.; Demetza, L.; Huber, S. Impact of mobile cross-platform development on CPU, memory and battery of mobile devices when using common mobile app features. *Procedia Comput. Sci.* **2020**, *175*, 189–196. [CrossRef]
16. Willocx, M.; Vossaert, J.; Naessens, V. A Quantitative Assessment of Performance in Mobile App Development Tools. In Proceedings of the 2015 IEEE International Conference on Mobile Services, New York, NY, USA, 27 June–2 July 2015.
17. Latif, M.; Lakhrissi, Y.; Nfaoui, E.H.; Es-sbai, N. Cross platform approach for mobile application development: A survey. In Proceedings of the 2016 International Conference on Information Technology for Organizations Development (IT4OD), Fez, Morocco, 30 March–1 April 2016.
18. Reetesh, V.G.; Prasann, A.V.; Pavan, H.B.; Manusmare, A. Design and Implementation of Android Base Mobile App for An Institute. In Proceedings of the International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT), Chennai, India, 3–5 March 2016.

19. Heitkotter, H.; Hanschke, S.; Majchrzak, T.A. Evaluating Cross-Platform Development Approaches for Mobile Applications, Web Information Systems and Technologies. In Proceedings of the 8th International Conference, WEBIST 2012, Porto, Portugal, 18–21 April 2012.
20. Fan, W.; Yang, J. Design and Implementation of Cross-platform Friends-Positioning Mobile APP Based on Phonegap and HTML5. In Proceedings of the 2017 2nd IEEE International Conference on Computational Intelligence and Applications, Chennai, India, 3–5 March 2016.
21. Huy, N.P.; Van Thanh, D. Developing Apps for Mobile Phones. In Proceedings of the 2012 7th International Conference on Computing and Convergence Technology (ICCT), Seoul, Korea, 3–5 December 2012.
22. Jobe, W. Native Apps vs. Mobile Web Apps. *Int. J. Interact. Mob. Technol.* **2013**, *7*, 27–32. [\[CrossRef\]](#)
23. Kho'i, F.H.; Jahid, J. *Comparing Native and Hybrid Applications with Focus on Features*; Faculty of Computing Blekinge Institute of Technology: Karlskrona, Sweden, 2016.
24. Elskovic, D. *A Comparison of Hybrid and Progressive Web Applications for the Android Platform*; Faculty of Computing, Blekinge Institute of Technology: Karlskrona, Sweden, 2020.
25. Pinto, C.M.; Coutinho, C. From Native to Cross-platform Hybrid Development. In Proceedings of the 2018 International Conference on Intelligent Systems (IS), Funchal, Portugal, 25–27 September 2018.
26. Pouryousef, S.; Rezaiee, M.; Chizari, A. Let me join two worlds! Analyzing the Integration of Web and Native Technologies in Hybrid Mobile Apps. In Proceedings of the 2018 17th IEEE International Conference on Trust, Security and Privacy in Computing and Communications/12th IEEE International Conference on Big Data Science and Engineering, New York, NY, USA, 1–3 August 2018.
27. Francese, R.; Gravino, C.; Risi, M.; Scanniello, G.; Tortora, G. Mobile App Development and Management: Results from a Qualitative Investigation. In Proceedings of the 2017 IEEE/ACM 4th International Conference on Mobile Software Engineering and Systems (MOBILESoft), Buenos Aires, Argentina, 22–23 May 2017.
28. Chandi, L.; Silva, C.; Martínez, D.; Gualotuña, T. Mobile application development process: A practical experience. In Proceedings of the 2017 12th Iberian Conference on Information Systems and Technologies (CISTI), Lisbon, Portugal, 21–24 June 2017.
29. Oluwatofunmi, A.; Chigozirim, A.; Nzechukwu, O.; Olawale, J.O. Dawning of Progressive Web Applications (PWA): Edging Out the Pitfalls of Traditional Mobile Development. *Am. Sci. Res. J. Eng. Technol. Sci. (ASRJETS)* **2020**, *68*, 85–99.
30. Bjørn-Hansen, A.; Majchrzak, T.A.; Grønli, T.-M. Progressive Web Apps: The Possible Web-native Unifier for Mobile Development. In Proceedings of the 13th International Conference on Web Information Systems and Technologies (WEBIST 2017), Porto, Portugal, 25–27 April 2017; pp. 344–351.
31. Dangkham, P. Mobile Augmented Reality on Web-Based for the Tourism Using HTML5. In Proceedings of the 2018 International Conference on Information Networking (ICOIN), Chiang Mai, Thailand, 10–12 January 2018.
32. Johansson, D.; Andersson, K. A Cross-Platform Application Framework for HTML5-based e-Services, Proceedings of the 11th Annual IEEE Consumer Communications & Networking Conference. In Proceedings of the 5th IEEE CCNC International Workshop on Mobility Management in the Networks of the Future World (MobiWorld 2014), Las Vegas, NV, USA, 10–13 January 2014.
33. Majchrzak, T.A.; Bjørn-Hansen, A.; Grønli, T.-M. Progressive Web Apps: The Definite Approach to Cross-Platform Development? In Proceedings of the 51st Hawaii International Conference on System Sciences, Hilton Waikoloa Village, HI, USA, 3 May 2018.
34. Jin, X.; Wang, L.; Luo, T.; Du, W. Fine-Grained Access Control for HTML5-Based Mobile Applications in Android. In Proceedings of the 16th Information Security Conference, Dallas, TX, USA, 13–15 November 2013.
35. El-Kaliouby, S.S.; Selim, S.; Yousef, A.H. Native Mobile Applications UI Code Conversion. In Proceedings of the 2021 16th International Conference on Computer Engineering and Systems (ICCES), Cairo, Egypt, 15–16 December 2021.
36. Ahuja, H.; Johari, R. Optimization of the Issues in the Migration from Android Native to Hybrid Application: Case Study of Student's Portal Application. In Proceedings of the 2014 International Conference on Advances in Computing, Communications and Informatics (ICACCI), Delhi, India, 24–27 September 2014.
37. Anderson, K.; Burford, O.; Emmerton, L. Mobile Health Apps to Facilitate Self-Care: A Qualitative Study of User Experiences. *PLoS ONE* **2016**, *11*, e0156164. [\[CrossRef\]](#)
38. Bessghaier, N.; Soui, M. Towards Usability Evaluation of Hybrid Mobile User Interfaces. In Proceedings of the 2017 IEEE/ACS 14th International Conference on Computer Systems and Applications, Hammamet, Tunisia, 30 October–3 November 2017.
39. Oliveira, W.; Torres, W.; Castor, F.; Ximenes, B.H. Native or Web? A Preliminary Study on the Energy Consumption of Android Development Models. In Proceedings of the 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering, Osaka, Japan, 14–18 March 2016.
40. Willocx, M.; Vossaert, J.; Naessens, V. Comparing performance parameters of mobile app development strategies. In Proceedings of the 2016 IEEE/ACM International Conference on Mobile Software Engineering and Systems (MOBILESoft), Austin, TX, USA, 14–22 May 2016.
41. Zhou, X.; Hu, W.; Liu, G.-P. React-Native Based Mobile App for Online Experimentation. In Proceedings of the 39th Chinese Control Conference, Shenyang, China, 27–29 July 2020.
42. Martinez, M.; Lecomte, S. Towards the quality improvement of cross-platform mobile applications. In Proceedings of the 2017 IEEE/ACM 4th International Conference on Mobile Software Engineering and Systems (MOBILESoft), Buenos Aires, Argentina, 22–23 May 2017.
43. Fortunato, D.; Bernardino, J. Progressive Web Apps: An alternative to the native mobile Apps. In Proceedings of the 2018 13th Iberian Conference on Information Systems and Technologies (CISTI), Cáceres, Spain, 13–16 June 2018.

-
44. Malavolta, I. Web-based Hybrid Mobile Apps: State of the Practice and Research Opportunities. In Proceedings of the 2016 IEEE/ACM International Conference on Mobile Software Engineering and Systems, Austin, TX, USA, 16–17 May 2016.
 45. Wasilewski, K.; Zabierowski, W. A Comparison of Java, Flutter and Kotlin/Native Technologies for Sensor Data-Driven Applications. *Sensors* **2021**, *21*, 3324. [[CrossRef](#)]
 46. A Complete History of Android. Available online: <https://www.techradar.com/news/phone-and-communications/mobile-phones/a-complete-history-of-android-470327> (accessed on 10 October 2021).
 47. FAQ | Kotlin. Available online: <https://kotlinlang.org/docs/faq.html#what-advantages-does-kotlin-give-me-over-the-java-programming-language> (accessed on 10 October 2021).
 48. Questions Tagged. Available online: <https://stackoverflow.com/questions/tagged/android> (accessed on 10 October 2021).
 49. The State of the Octoverse. Available online: <https://octoverse.github.com/projects> (accessed on 10 October 2021).
 50. What Is Ionic. Available online: <https://ionicframework.com/what-is-ionic> (accessed on 10 October 2021).
 51. Questions Tagged. Available online: <https://stackoverflow.com/questions/tagged/ionic-framework> (accessed on 10 October 2021).